



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search prioritizes nodes only according to the path cost from the start node, $g(n)$. It always expands the node with the lowest accumulated cost so far. A* Search uses both the path cost and an estimate of the remaining cost to the goal. It evaluates nodes using $f(n) = g(n) + h(n)$, where $h(n)$ is the heuristic estimate from the current node to the goal. Therefore, A* is more goal-directed than UCS because it uses information about where the goal is located.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is admissible in this grid because the agent can only move Up, Down, Left, and Right. The Manhattan Distance calculates the minimum number of horizontal and vertical movements required to reach the goal when there are no obstacles. Walls may force the actual route to become longer, but Manhattan Distance will never overestimate the true shortest path cost. Therefore, it is an admissible heuristic.

If the heuristic was not admissible, it could overestimate the real cost of reaching the goal. In that case, A* might ignore a path that actually leads to the optimal solution and choose a more expensive path instead. As a result, A* would no longer be guaranteed to find the optimal path.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

If diagonal movement was allowed and a diagonal move had the same cost as a horizontal or vertical move, Manhattan Distance would no longer be admissible. This is because Manhattan Distance assumes that horizontal and vertical movements must be performed separately. With diagonal movement, the agent could reach the goal using fewer moves than the Manhattan Distance predicts, causing the heuristic to overestimate the true path cost.

For an 8-way movement grid, a more suitable heuristic is Chebyshev Distance when horizontal, vertical, and diagonal movements all have the same cost. Chebyshev Distance can be calculated as:

$$\max(|x_1 - x_2|, |y_1 - y_2|)$$

If diagonal movements instead have a cost such as $\sqrt{2}$ while horizontal and vertical moves cost 1, Octile Distance would be more appropriate. Euclidean Distance can also provide a valid straight-line estimate, but Chebyshev or Octile Distance better matches typical 8-way grid movement costs.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A stronger heuristic could consider both the distance to the nearest food item and the cost of connecting all remaining food items. One approach is to calculate the distance from the agent to the closest remaining food and then add the cost of a Minimum Spanning Tree over all remaining food positions.

The heuristic could be written conceptually as:

$$h(n) = \text{distance to closest food} + \text{MST cost of all remaining food}$$

The Minimum Spanning Tree estimates the minimum total distance needed to connect all remaining food items without requiring a specific visiting order. This gives A* more information than simply considering the nearest food item. It provides a stronger estimate of the remaining work required to collect all

food while still avoiding unnecessary overestimation if suitable admissible distance measures are used.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING